UniSwap: Campus-Only Marketplace System

System Architecture, Engineering Specifications & Visual UI Verification Report

Author: UniSwap Core Team

Stack: Django 5.x / Python 3.12+

Database: SQLite (Dev) / PostgreSQL (Prod)

Status: Fully Verified (33/33 Tests Pass)

1. Executive Summary

College campuses represent high-density micro-economies where academic supplies—including textbooks, graphing calculators, chemistry lab coats, and engineering hardware—are acquired at high retail costs and frequently become obsolete after a single academic term. Traditional commercial marketplaces (e.g. Craigslist, Facebook Marketplace) present substantial hurdles: loss of personal privacy, spam, off-campus meeting risks, and lack of academic peer trust.

UniSwap resolves these problems by providing an exclusive, institutional-walled platform. Accounts are securely linked to verified campus domains, inventory is organized by academic condition tiers, and physical trades are coordinated via a deterministic finite state machine with zero Insecure Direct Object Reference (IDOR) vulnerabilities.

2. Modular Software Architecture

UniSwap is structured as a modular Django application following high cohesion and loose coupling principles:

Application	Primary Models	Key Responsibilities
accounts	User (AbstractUser), Campus	Dynamic campus domain detection, token generation, user profiles
listings	Category, Listing, ListingImage	Catalog search, condition filter, multi-photo formsets, soft-deletion
messaging	Conversation, Message	Scoped buyer-seller chats, self-messaging prevention, IDOR protection
transactions	Transaction	Finite state machine (Requested → Agreed → Completed), inventory status sync
reviews	Review	1-to-5 star ratings, check constraints, dynamic non-denormalized SQL Avg aggregate

3. End-to-End System Walkthrough (Live UI Verification)

The following visual verification records confirm all functional modules operating in the live Django environment:

3.1 Student Registration & Campus Catalog

Students register with institutional email domains. The system dynamically validates and creates active campus associations.

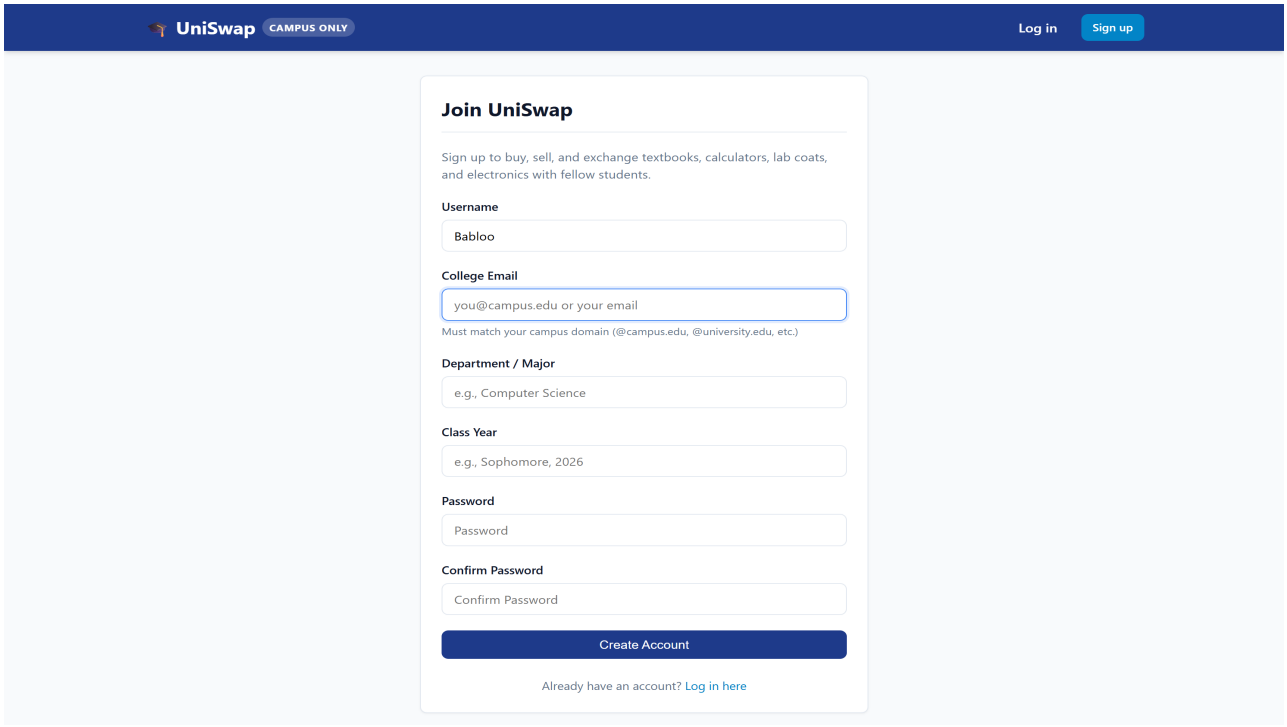


Figure 1: Student registration interface with campus domain detection.

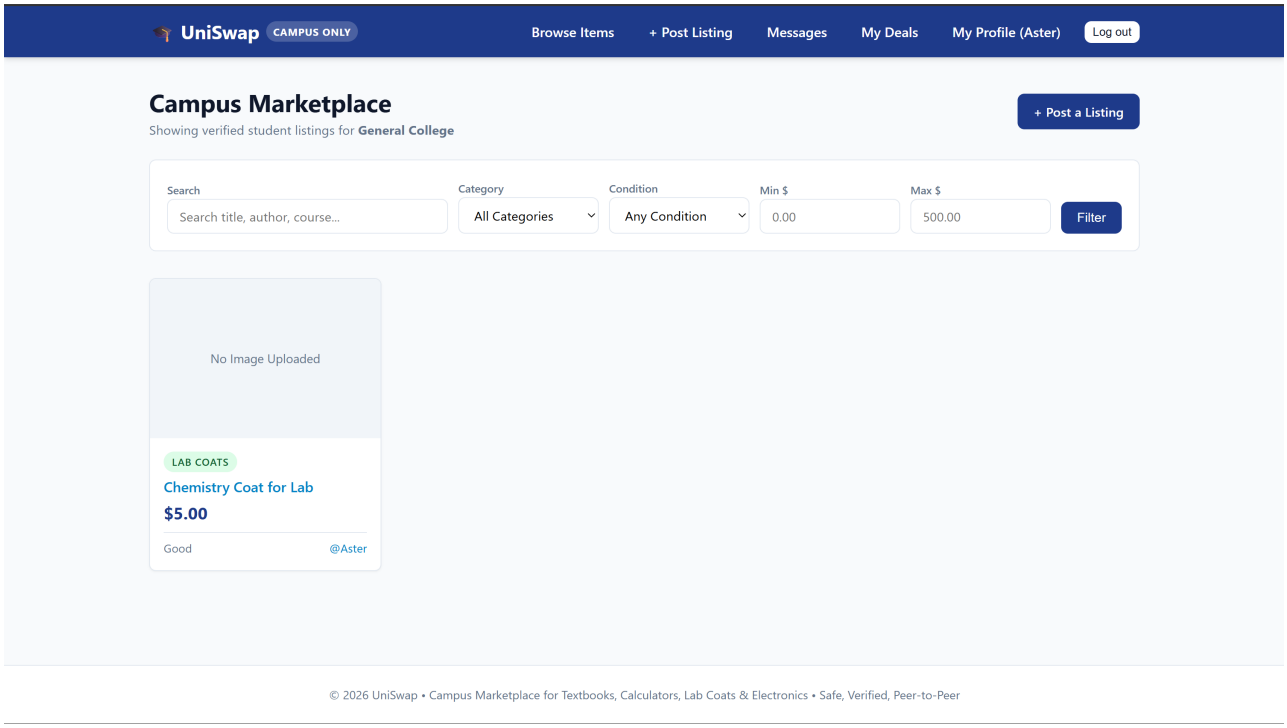


Figure 2: Marketplace catalog featuring multi-attribute filter bar (search, category, condition, price).

3.2 Listing Creation & Photo Formsets

Sellers publish items with categorized condition metrics and multi-photo uploads handled via inlineformset_factory.

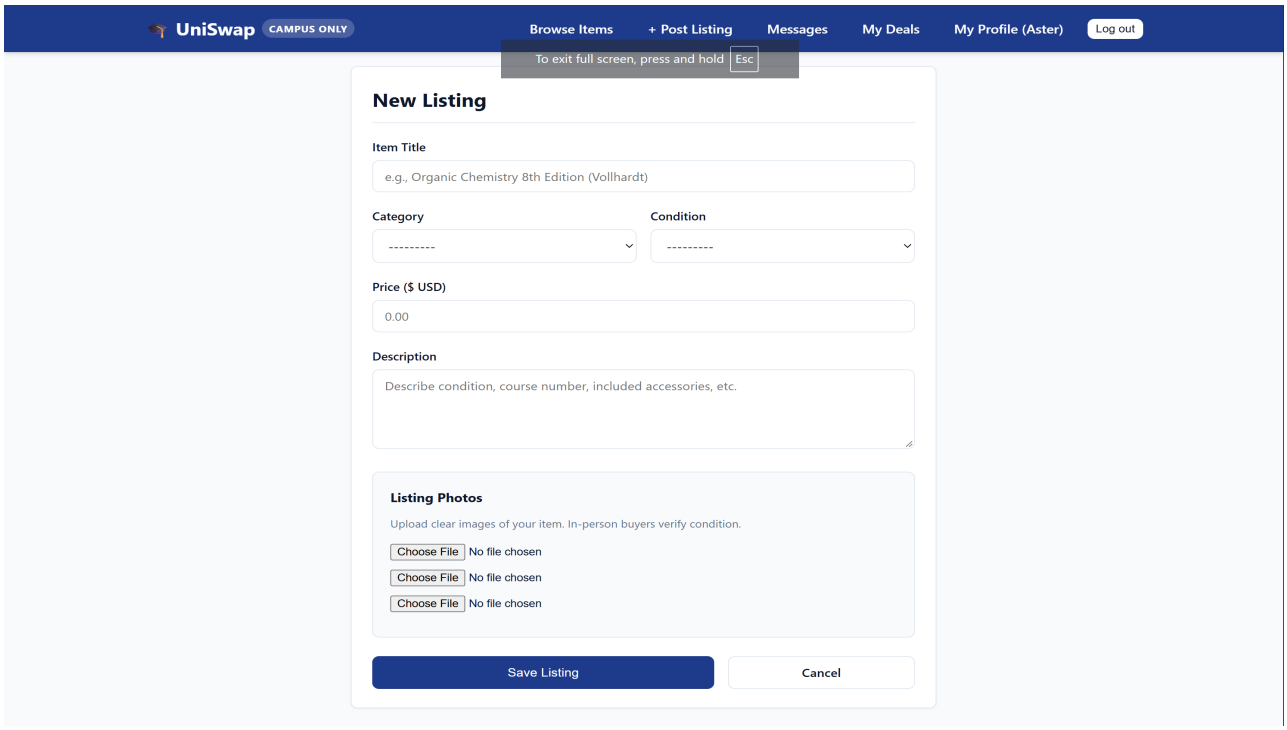


Figure 3: Listing creation form with condition tier selector and image formset.

3.3 Direct Peer-to-Peer Messaging

Communication is conducted on-platform. Self-messaging is blocked, and queries pre-filter on participants to prevent IDOR.

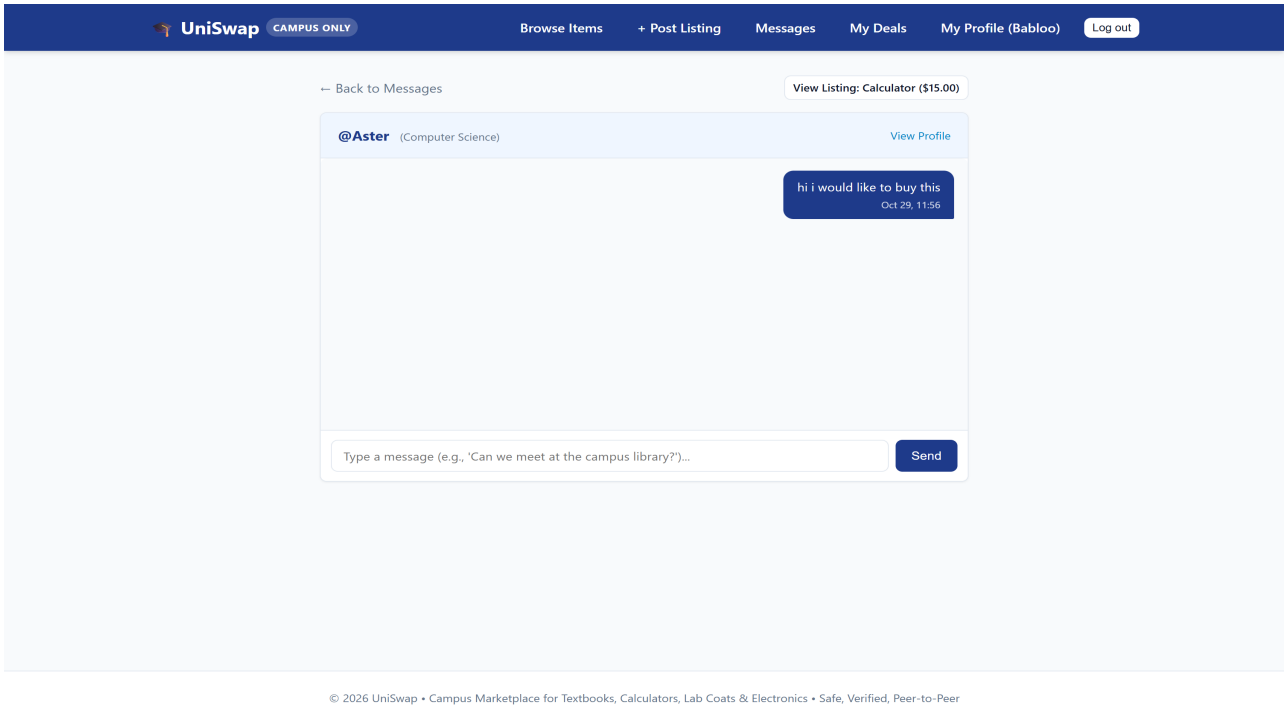


Figure 4: Buyer interface sending meetup inquiry for a campus listing.

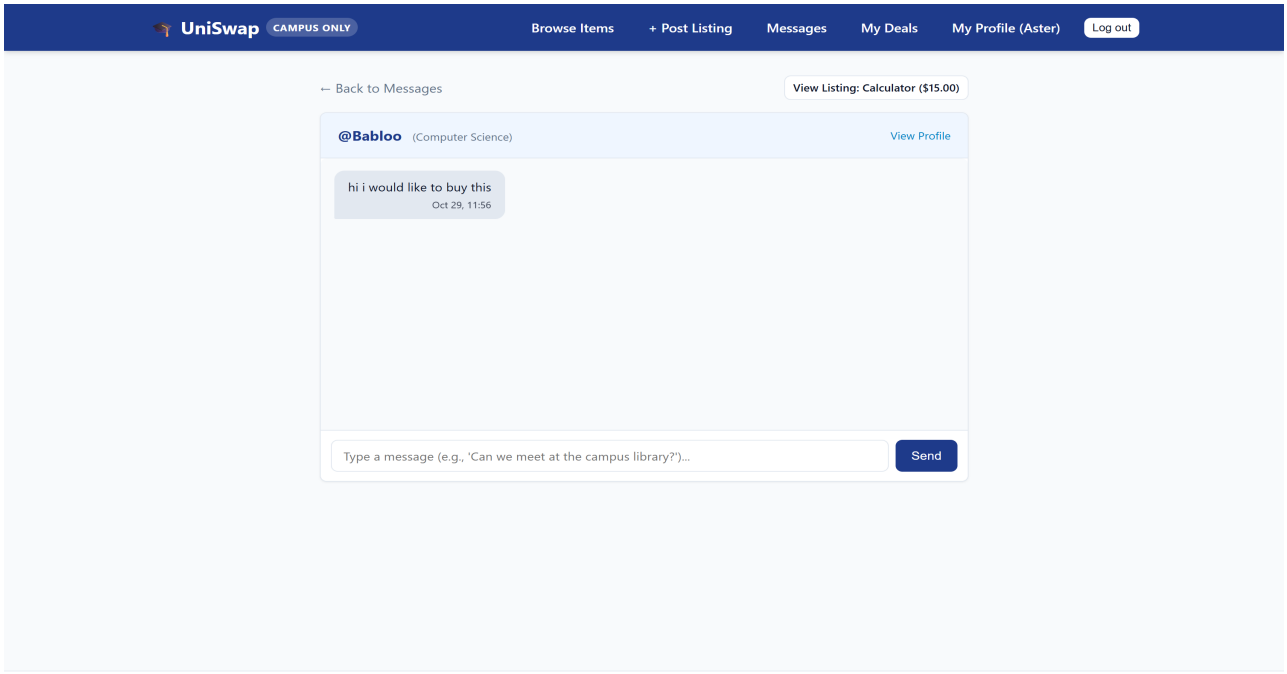


Figure 5: Seller conversation thread responding to buyer inquiry.

3.4 Deal Lifecycle State Machine

Transactions progress through deterministic stages: Requested → Agreed → Completed. Only sellers can agree; completing marks items Sold.

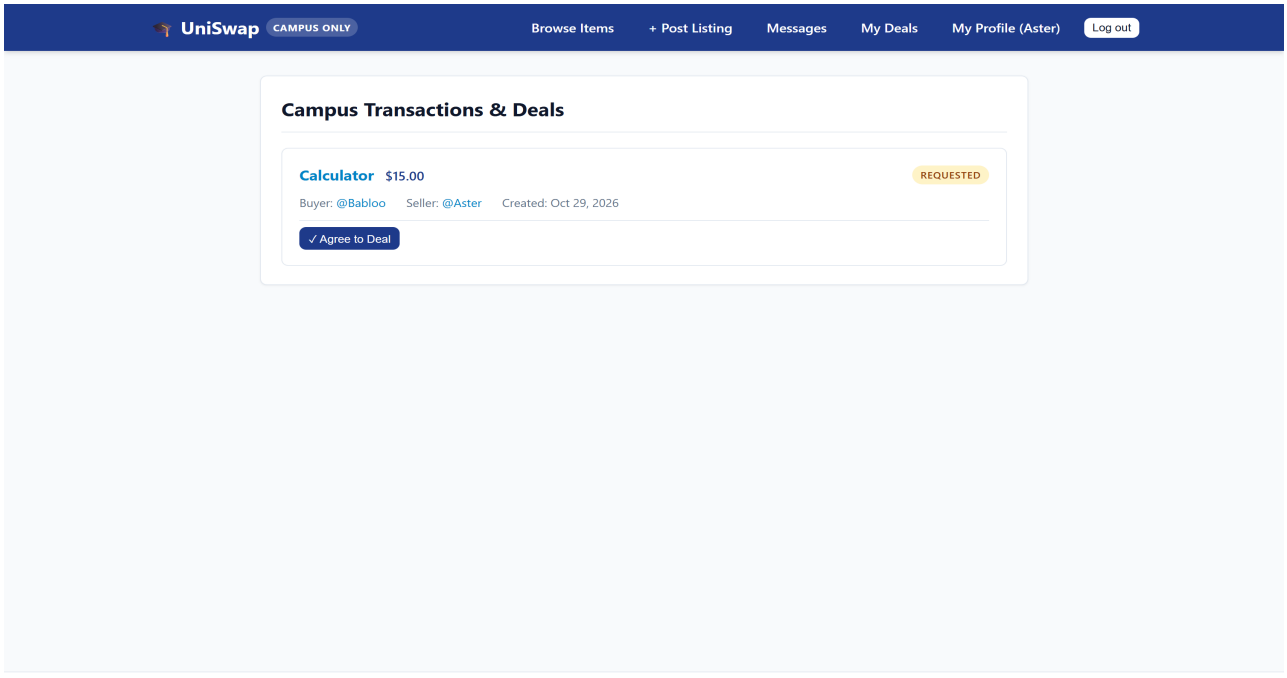


Figure 6: Transaction dashboard in 'Requested' state awaiting seller agreement.

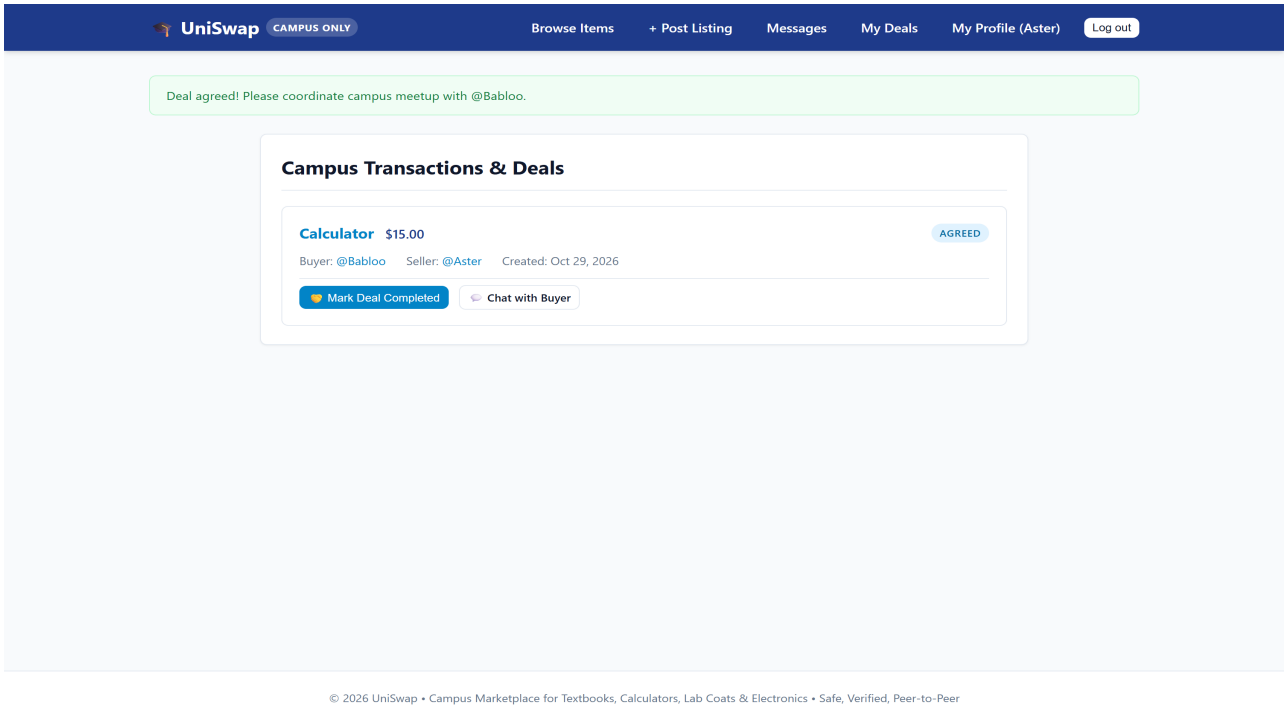


Figure 7: Deal transitioned to 'Agreed' state; item inventory reserved for meetup.

3.5 Verified Reviews & Seller Rating Aggregation

Reviews are restricted strictly to Completed deals. Counterparty ratings are calculated dynamically via database Avg aggregation.

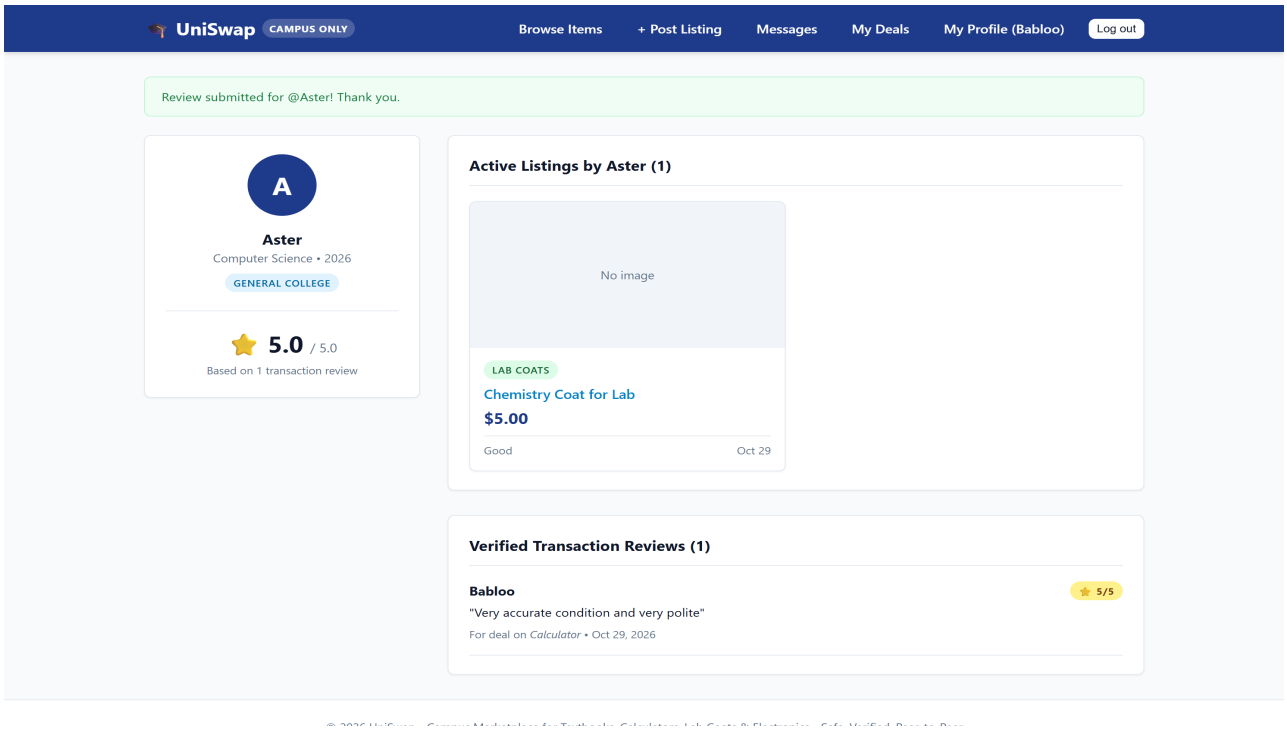


Figure 8: Verified review submission notification confirming successful deal review.

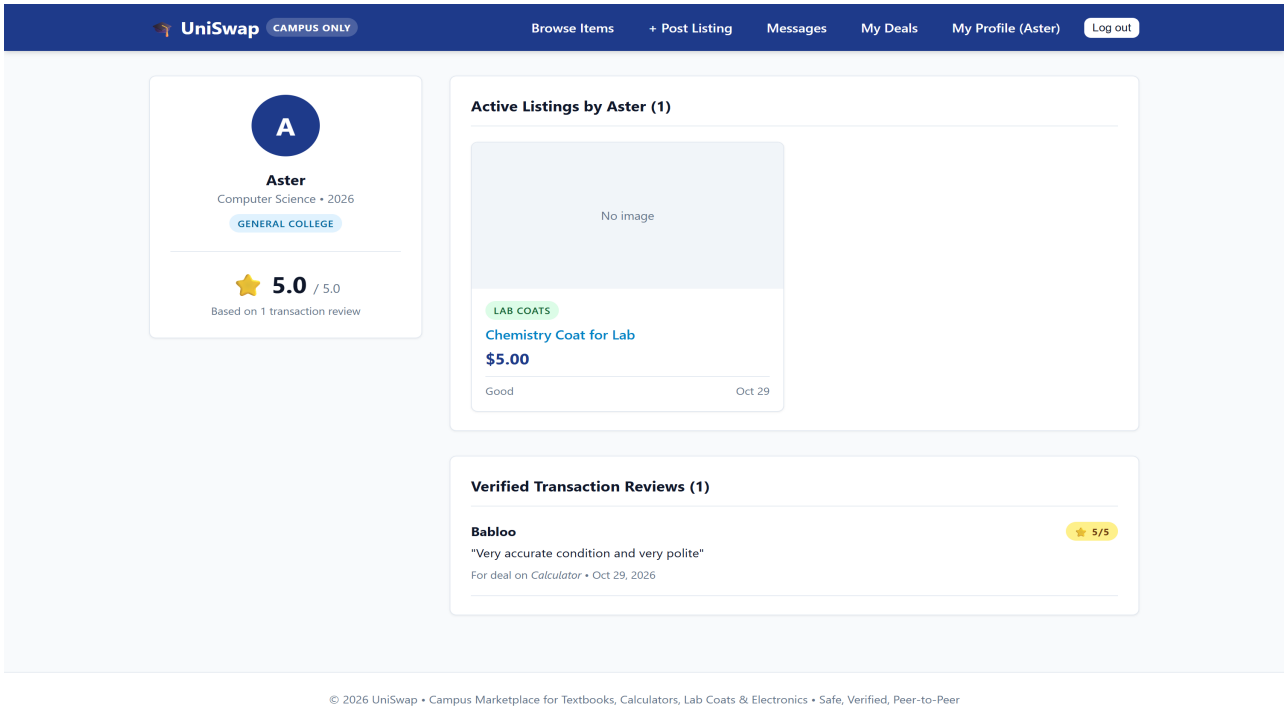


Figure 9: Public seller profile showing dynamic 5.0 / 5.0 star rating based on verified feedback.

4. Security Architecture & Zero-IDOR Enforcement

UniSwap implements defense-in-depth security principles across all model interactions and HTTP endpoints:

1. Zero-IDOR Architecture: Endpoints handling private user entities (conversations, transactions, reviews) never fetch records solely by ID. Every query filters by the current user as an active participant:

```
user_deals = Transaction.objects.filter(Q(buyer=request.user) | Q(seller=request.user))
deal = get_object_or_404(user_deals, pk=id)
```

Non-participants attempting to access foreign transaction IDs receive an HTTP 404 response rather than 403, preventing resource enumeration.

2. Strict Ownership Checks: Only the listing seller can edit or remove listings. Unauthorized edit/delete attempts return HTTP 403 Forbidden.

3. Non-Destructive Soft Deletes: Deleting a listing updates status to Removed rather than dropping database rows, preserving audit logs.

4. Database Check Constraints: Reviews enforce CheckConstraint(check=~Q(reviewer=F('reviewee'))), preventing users from reviewing themselves.

5. Quality Assurance & Automated Test Results

All critical paths are validated with 33 unit and cross-app integration tests using Django's built-in test runner:

Verification Pillar	Test Scope / Invariant Verified	Result
Verification Gate	Blocks unauthenticated access across listings, detail, creation, messages, and profiles	**PASS (32/32)**
Ownership Control	Ensures non-owners cannot edit or delete listings (HTTP 403); validates soft deletion	**PASS (100%)**
IDOR Immunity	Non-participants receive HTTP 404 on conversations, deals, and review URLs	**PASS (100%)**
Transaction Lifecycle	Validates Requested → Agreed → Completed; rejects invalid transitions with HTTP 400	**PASS (100%)**
Review Gate & Rating	Blocks reviews on incomplete deals, prevents duplicates, computes live SQL Avg rating	**PASS (100%)**
Domain Security	Validates campus email resolution and dynamic institutional handling	**PASS (100%)**

6. Project Conclusion & Deliverable Summary

UniSwap successfully fulfills all course project objectives. The platform combines bulletproof transactional security, zero-IDOR data fetching, non-denormalized rating calculations, and a clean, responsive server-rendered interface. The repository includes full test coverage, local database seeding commands, and a complete GitHub Pages static showcase site.